信号与系统课程设计

快速傅里叶变换及 Chirp-Z 变换

2024.6.28

Contents

1 引言 **3**

2 快速傅里叶变换 **3**

2.1 符号约定 3

2.2 多项式与离散傅里叶变换 3

2.2.1 引言 3

2.2.2 多项式 3

2.2.3 离散傅里叶变换 4

2.3 快速傅里叶变换 5

2.3.1 基本原理 5

2.3.2 位逆序置换和蝶形运算 6

3 多项式乘法的快速实现 **7**

3.1 引言 7

3.2 一般情况下的多项式乘法 7

3.3 一些特殊情景下的多项式乘法 7

3.3.1 实系数多项式乘法 7

3.3.2 （实系数）任意模数多项式乘法 8

4 Chirp-Z 变换 **10**

5 测试及结果 **12**

6 Appendix: Source Code (in C++) **13**

6.1 快速傅里叶变换及利用 FFT 的快速卷积 13

6.2 Chirp-Z 变换 15

1 引言

本文主要讨论快速傅里叶变换及利用快速傅里叶变换的卷积实现。在第二节中，将对快速傅里叶变换的原理及实现展开讨论，为此我们将引入多项式的概念，并从多项式的视角出发讨论傅里叶变换。在第三节中，将讨论快速傅里叶变换在卷积即多项式乘法上的具体应用，并对一些特殊情形进行额外的拓展。在第四节，我们将讨论任意长度的傅里叶变换的一种实现方式，即 Chirp-Z 变换。

2 快速傅里叶变换

2.1 符号约定

- $f[n]$：数列 f 的第 n 项。
- $F[n]$（或 $[x^n]F(x)$）：多项式 $F(x)$ 中 x^n 一项的系数。
- 多项式乘法：$F(x)*G(x)=\sum_{i=0}\sum_{j=0}F[i]G[j]x^{i+j}$，则有 $(F*G)[n]=\sum_{i=0}^{n}F[i]G[n-i]$。

2.2 多项式与离散傅里叶变换

2.2.1 引言

傅里叶变换是信号处理中的常用工具，它将一个时域信号进行正交基分解转换为频域信号，从而在处理信号（主要是处理两个信号的卷积）时能够更简便。然而，对于离散傅里叶变换，我们有一个更直观的理解角度——即从多项式的角度去看待离散傅里叶变换。在这一角度下，我们可以将其视为多项式求值/插值的过程，从而更容易得出具体的实现方式。在这一小节，我们将首先介绍多项式的概念，并通过多项式的基本运算自然的引出离散傅里叶变换。

2.2.2 多项式

Definition 2.1 (n 次多项式)**.** *定义 $F(x)=\sum_{i=0}^{n-1}F[i]x^i$ 为 n 次多项式，其中 n 为该多项式的***次数***，也称为多项式的***度***，记为 $\deg f$，$F[i]$ 为该多项式第 i 项系数，有时也记为 $[x^i]F(x)$。*

有关多项式的严格定义需要借助抽象代数中环论的知识，这与本文的主题无关，因此这里仅对这一数学概念做一个粗略的定义。

Definition 2.2 (多项式乘法)**.** *定义两个多项式的乘法（有时也称多项式的卷积）运算 $*$ 为：$F(x)*G(x)=\sum_{i=0}\sum_{j=0}F[i]G[j]x^{i+j}$。*

该定义就是我们通常使用的多项式乘法的定义。此外，多项式的加法也是通常使用的对应项系数直接相加，不再单独定义。

对于多项式而言，最基础的运算是多项式的乘法和加法。假设我们在处理一个 n 次多项式和一个 m 次多项式，显而易见的，多项式加法的时间复杂度为 $O(n+m)$，但如果按照定义计算多项式乘法，时间复杂度为 $O(nm)$，这成为了相关操作的复杂度瓶颈。因此在下文中，我们重点关注的是如何快速计算多项式乘法。

我们不加证明的给出一个定理：

Theorem 2.1. 互不相同的 n+1 个点的点值唯一确定一个 n 次多项式。

该定理可以由代数基本定理推出，由于这不是本文主题，不再赘述。

因此，假设我们知道了 $n+1$ 个互不相同的点 $x_0, x_1, \ldots, x_n$ 以及多项式在这些点的点值 $F(x_0), F(x_1), \ldots, F(x_n)$，我们就可以通过这些点值还原出多项式的系数。据此，我们引入多项式的点值表示法：

Definition 2.3 (点值表示法)**.** 一个 n 次多项式的点值表示法为 $n+1$ 个横坐标互不相同的点 $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$。

在多项式的点值表示下，多项式乘法的计算将变得更加简洁：假设我们有一个 n 次多项式和一个 m 次多项式，那么我们知道他们卷积的结果是一个 n+m-1 次多项式，如果我们知道这两个多项式在 n+m 个相同的点的点值，并将对应点的点值相乘，那么得到的将是这两个多项式卷积得到的 n+m-1 次多项式的点值表示。由此，多项式乘法的复杂度被降低到了 $O(n+m)$。我们需要做的就是如何得到其点值表示。我们称这样的将多项式系数表示转换为点值表示的过程为**求值**，其逆过程即由点值表示转换为系数表示的过程为**插值**。

2.2.3 离散傅里叶变换

多项式的求值如果按照定义计算，时间复杂度将是 $O(nm)$，这就与多项式乘法的朴素实现相同了。然而如果注意到多项式乘法计算过程对具体的点没有做出限制，那么我们可以选择一些特殊的点进行求值，从而加快求值的速度。为此，我们引入离散傅里叶变换。在这一角度下，我们可以将离散傅里叶变换看做对给定的系数多项式在单位根处求值。

Definition 2.4 (离散傅里叶变换)**.** 设一 n 次多项式 $f(x) = \sum_{i=0}^{n-1} f[i]x^i$，定义其离散傅里叶变换为 $F[k] = \sum_{i=0}^{n-1} f[i]e^{-j\frac{2\pi}{n}ki}$，其中 k 的取值从 *0* 到 *n-1*，$F[k]$ 也即 $f(x)$ 在点 $x = e^{-j\frac{2\pi}{n}k}$ 处的点值。下文中的 *DFT* 为离散傅里叶变换的缩写。

另外，根据上述定义，我们还可以推出 DFT 的反变换的表达式：

Definition 2.5 (离散傅里叶反变换)**.** 设一 n 次多项式 $f(x) = \sum_{i=0}^{n-1} f[i]x^i$，其离散傅里叶变换为 $F[k] = \sum_{i=0}^{n-1} f[i]e^{-j\frac{2\pi}{n}ki}$，则其离散傅里叶反变换为 $f[i] = \frac{1}{n}\sum_{k=0}^{n-1} F[k]e^{j\frac{2\pi}{n}ki}$。下文中的 *IDFT* 为离散傅里叶反变换的缩写。

证明如下：

$$
\begin{aligned}
\text{右边} &= \frac{1}{n}\sum_{k=0}^{n-1}(\sum_{t=0}^{n-1} f[t]e^{-j\frac{2\pi}{n}kt})e^{j\frac{2\pi}{n}ki} \\
&= \frac{1}{n}\sum_{t=0}^{n-1} f[t]\sum_{k=0}^{n-1} e^{j\frac{2\pi}{n}ki-j\frac{2\pi}{n}kt} \\
&= \frac{1}{n}\sum_{t=0}^{n-1} f[t]\sum_{k=0}^{n-1} e^{j\frac{2\pi}{n}k(i-t)} \\
&= \frac{1}{n}(f[i]\sum_{k=0}^{n-1} e^{0} + \sum_{0\le t<n, t\ne i} f[t]\frac{e^0(1-e^{j\frac{2\pi}{n}(i-t)n})}{1-e^{j\frac{2\pi}{n}(i-t)}}) \\
&= \frac{1}{n}(nf[i] + \sum_{0\le t<n, t\ne i} f[t]\frac{1-e^{(i-t)2\pi j}}{1-e^{j\frac{2\pi}{n}(i-t)}}) \\
&= \frac{1}{n}(nf[i] + \sum_{0\le t<n, t\ne i} f[t]\frac{1-1}{1-e^{j\frac{2\pi}{n}(i-t)}}) \\
&= f[i] = \text{左边}
\end{aligned}
$$

可以发现，DFT 就是对给定的 n 次多项式，求它在所有的 n 次单位根处的点值，而 IDFT 则是其逆过程。这正好与上文快速计算多项式乘法的思路对应。我们首先通过 DFT 求得待卷积多项式的点值表示，将对应点值相乘后再通过 IDFT 得到结果多项式的系数表示。然而，通过定义式朴素的计算离散傅里叶变换的时间复杂度为 $O(n^2)$，此时多项式乘法的复杂度并未得到改进。为此，我们引入下文的快速傅里叶变换来加速这一过程。

2.3 快速傅里叶变换

2.3.1 基本原理

以下内容中的快速傅里叶变换（FFT）均指基-2 快速傅里叶变换。它只能接受一个次数为 2 的幂次的多项式，并求出在对应的单位根处的点值。

FFT 的基本思想是分治，具体的，记多项式的次数为 n，$\omega_n = e^{-j\frac{2\pi}{n}}$ 为 n 次单位根（的共轭，事实上，其性质与 n 次单位根完全相同），待求值多项式 $f(x) = \sum_{i=0}^{n-1} f[i]x^i$，那么我们把多项式分为偶次项 $g(x) = \sum_{i=0}^{\frac{n}{2}-1} f[2i]x^i$ 和奇次项 $h(x) = \sum_{i=0}^{\frac{n}{2}-1} f[2i+1]x^i$，有：

$$f(x) = g(x^2) + xh(x^2)$$

注意到 ω_n 是偶数次单位根，那么有性质：$\omega_n^{2k} = \omega_{\frac{n}{2}}^k$ 且 $\omega_n^k = -\omega_n^{k+\frac{n}{2}}$，因此 $g((\omega_n^k)^2) = g((\omega_n^{k+\frac{n}{2}})^2)$，对 h 也是同样的，因此有：

$$
\begin{aligned}
f(\omega_n^k) &= g((\omega_n^k)^2) + \omega_n^k h((\omega_n^k)^2) \\
&= g(\omega_n^{2k}) + \omega_n^k h(\omega_n^{2k}) \\
&= g(\omega_{\frac{n}{2}}^k) + \omega_n^k h(\omega_{\frac{n}{2}}^k)
\end{aligned}
$$

和

$$\begin{aligned} f(\omega_n^{k+\frac{n}{2}}) &= g((\omega_n^{k+\frac{n}{2}})^2) + \omega_n^{k+\frac{n}{2}} h((\omega_n^{k+\frac{n}{2}})^2) \\ &= g((\omega_n^k)^2) - \omega_n^k h((\omega_n^k)^2) \\ &= g(\omega_n^{2k}) - \omega_n^k h(\omega_n^{2k}) \\ &= g(\omega_{\frac{n}{2}}^k) - \omega_n^k h(\omega_{\frac{n}{2}}^k) \end{aligned}$$

当我们求出了 $g(\omega_{\frac{n}{2}}^0), g(\omega_{\frac{n}{2}}^1), \ldots, g(\omega_{\frac{n}{2}}^{\frac{n}{2}-1})$ 和 $h(\omega_{\frac{n}{2}}^0), h(\omega_{\frac{n}{2}}^1), \ldots, h(\omega_{\frac{n}{2}}^{\frac{n}{2}-1})$ 后，便可由上两式求出 $f(\omega_n^0), f(\omega_n^1), \ldots, f(\omega_n^{n-1})$，因此只需递归处理两个规模减半的 DFT 即可。时间复杂度 $T(n) = 2T(\frac{n}{2}) + O(n)$，由主定理，我们有：

Theorem 2.2. 快速傅里叶变换的时间复杂度为 $O(nlogn)$。

另外，由于 DFT 和 IDFT 形式上的对偶性（或者注意到 $\omega_n^i = \omega_n^{i-n} = \omega_n^{-(n-i)}$），IDFT 的结果可以通过 DFT 的结果得到，因此不再赘述。

2.3.2 位逆序置换和蝶形运算

在前述 FFT 的基本原理中，我们讨论了其递归实现。在实际应用中，出于常数上的考虑，常使用倍增的方法实现。为此，我们介绍 FFT 的倍增实现的基础，位逆序置换和蝶形运算。

以长度（次数）为 8 的 FFT 为例。模拟拆分的过程，观察待求值多项式 f 的系数：

0 次拆分：$\{f[0], f[1], f[2], f[3], f[4], f[5], f[6], f[7]\}$。

1 次拆分：$\{f[0], f[2], f[4], f[6]\}, \{f[1], f[3], f[5], f[7]\}$。

2 次拆分：$\{f[0], f[4]\}, \{f[2], f[6]\}, \{f[1], f[5]\}, \{f[3], f[7]\}$。

3 次拆分：$\{f[0]\}, \{f[4]\}, \{f[2]\}, \{f[6]\}, \{f[1]\}, \{f[5]\}, \{f[3]\}, \{f[7]\}$。

如果我们把最后得到的系数序列编号和位置以二进制的形式写出，则：

$$\{(000)_2, (100)_2, (010)_2, (110)_2, (001)_2, (101)_2, (011)_2, (111)_2\}$$
$$\{(000)_2, (001)_2, (010)_2, (011)_2, (100)_2, (101)_2, (110)_2, (111)_2\}$$

从中可见 f 的第 i 项系数 $f[i]$ 在拆分后的位置，就是将 i 的二进制表示翻转后得到的数。这是因为，考虑每一次拆分的过程，我们按奇偶将序列分为两部分。第 i 次二分时，奇偶对应于从低到高第 i 个二进制位的取值，若取值为 0，则是在偶数项，反之则在奇数项。我们将偶数项放到左边，奇数项放到右边，就相当于以从低到高第 i 个二进制位为关键字进行排序，因此拆分的过程可以看做是以从低到高第一个二进制位为第一关键字，第二个二进制位为第二关键字，…，第 n 个二进制位为第 n 关键字，对所有项进行排序，自然，这就相当于将每一项位置的二进制位翻转后，递增的排序。

因此，我们可以省去递归拆分的步骤，将待求值多项式的系数位置的二进制位翻转后重新排列，这一操作被称为位逆序置换。

另一方面，由于系数已经提前按顺序排列好，$g(\omega_{\frac{n}{2}}^k)$ 将储存在第 k 个位置，$h(\omega_{\frac{n}{2}}^k)$ 将储存在第 $\frac{n}{2}+k$ 个位置，而运算完成后 $f(\omega_n^k)$ 将储存在第 k 个位置，$f(\omega_n^{k+\frac{n}{2}})$ 将储存在第 $\frac{n}{2}+k$ 个位置，因此求值时可以直接在原位置上进行覆写而无需额外的空间。这样的操作称为蝶形运算。

3 多项式乘法的快速实现

3.1 引言

在本节中，我们将讨论如何利用上节介绍的快速傅里叶变换计算多项式乘法。我们首先给出一个普遍适用的做法，同时也是下文 Chirp-Z 变换的基础。然后我们将拓展两类特殊情况下的多项式乘法算法，分别是实系数多项式乘法，和任意模数多项式乘法。

3.2 一般情况下的多项式乘法

在上一节中，我们提到过，在计算一个 n 次多项式 f 和一个 m 次多项式 g 的乘法时，我们可以求出这两个多项式的 n+m 个点值，然后将点值直接相乘后再插值得到待求的 n+m-1 次多项式。然而，我们也知道，快速傅里叶变换只接受长度是 2 的整数次幂的多项式。为了解决这一矛盾，一个基本的做法是：令 $l = \inf\{n|2^n \geq n+m\}$，我们可以将这两个多项式分别写成 l 次多项式 f' 和 g'，使得 $f'[i] = \begin{cases} f[i] & i < n \\ 0 & n \leq i < l \end{cases}, g'[i] = \begin{cases} g[i] & i < m \\ 0 & m \leq i < l \end{cases}$。显然，$f' * g' = f * g$，而 f' 和 g' 的次数为 2 的整数次幂。这样，通过计算 f' 和 g' 这两个次数为 l 的多项式乘法，我们可以得到待求的结果。

这一做法需要计算 f' 和 g' 的 DFT，然后计算二者的点值相乘后计算一次 IDFT，因此总共需要计算 3 次 DFT 的时间。

3.3 一些特殊情景下的多项式乘法

3.3.1 实系数多项式乘法

如前一小节中所讨论的，对于一般情况的多项式乘法，存在一种 3 次 DFT 的快速算法。这里，我们讨论在多项式系数均为实数的情况下，是否可以减少 DFT 次数以加快计算速度。

对于实系数的多项式，其虚部恒为 0。然而在快速傅里叶变换中，我们的求值和插值均是在复数域 $\mathbb{C}$ 中进行的，因此可以考虑将虚部加以利用。具体的，假设我们要计算实系数多项式 a 和实系数多项式 b 的乘法，考虑构造一个复系数多项式 c，满足 $c[n] = a[n] + j \cdot b[n]$，其中 j 为虚数单位。考虑 c 与自身的卷积：

$$\begin{aligned}
(c * c)[n] &= \sum_{i+k=n} c[i] \times c[k] \\
&= \sum_{i+k=n} (a[i] + j \cdot b[i]) \times (a[k] + j \cdot b[k]) \\
&= \sum_{i+k=n} (a[i]a[k] - b[i]b[k]) + j \cdot (a[i]b[k] + b[i]a[k]) \\
&= \sum_{i+k=n} (a[i]a[k] - b[i]b[k]) + 2j \cdot (a[i]b[k])
\end{aligned}$$

注意到其虚部恰是 a 和 b 相乘的两倍，因此 $a * b = \mathbf{Im}\{\frac{c*c}{2}\}$。

这样，我们只需要对 c 做一次 DFT，再做一次 IDFT 即可得到 a*b 的结果。DFT 次数被我们减少到了 2 次，这大约可以将用时减少 $\frac{1}{3}$。

3.3.2 （实系数）任意模数多项式乘法

注意到在快速傅里叶变换中，我们的求值和插值均是在复数域 $\mathbb{C}$ 中进行的，在实际的实现中需要使用到以浮点数为基础的复数，因而不可避免的涉及到浮点数精度的问题。然而，在一些实际应用中不需要求出多项式乘法具体的值，而是需要求出对给定模数取模的结果。

假若给定的模数 p 满足某些特殊性质，具体的，次数 n 可除 p-1 且 p 是质数，那么我们可以用 p 的原根代替单位根，做快速数论变换。然而这是一个特殊情形，在更一般的情况下这一做法并不可行，例如次数 $n = 10^6$，模数 $p = 10^9 + 7$。

另一方面，在这一例中做 FFT 然后将点值直接相乘也是不可行的，这是因为相乘后的值域最大可以达到 $p^2 n \approx 10^{9+9+5} = 10^{23}$，而双精度浮点数的精度最大只能达到约 $2^{52} \approx 10^{15}$，是达不到精度要求的。

为了解决这一问题，一般有两种方法。第一种方法基于快速数论变换，我们可以选取三个 10^9 级别的特殊模数 p_1, p_2, p_3，分别计算出对 p_1, p_2, p_3 取模的结果，然后利用中国剩余定理得到对给定的模数 p 取模的结果。这种做法需要做三次多项式乘法，因此需要 9 次 DFT，在不经过特殊优化的情况下效率较低。

第二种方法，也是本文提出的方法，则是拆系数 FFT。这种方法的原理如下：

假设给定的两个多项式为 $a(x) = \sum_{i=0} a[i]x^i$ 和 $b(x) = \sum_{i=0} b[i]x^i$，我们构造多项式 $a0, a1, b0, b1$ 满足：$a = a0 + base \times a1$，$b = b0 + base \times b1$，其中 $base \approx \sqrt{p}$ 为一正整数，$base \times f = \sum_{i=0}(base \times f[i])x^i$，那么我们有：

$$\begin{aligned} a * b &= (a0 + base \times a1) * (b0 + base \times b1) \\ &= a0 * b0 + (a0 * b1 + a1 * b0) \times base + a1 * b1 \times base^2 \end{aligned}$$

因此我们只需计算出 $a0 * b0, a0 * b1, a1 * b0, a1 * b1$ 这四个卷积，即可得到 $a * b$ 的卷积。而这四个卷积的值域约为 $\sqrt{p}^2 n = pn \approx 10^{9+5} = 10^{14}$，正好在双精度浮点数的精度范围内，不会出现精度问题。

进一步的，为了加速计算这四个卷积，我们令：

$$\begin{aligned} f &= a0 + ja1 \\ g &= a0 - ja1 \\ h &= b0 + jb1 \end{aligned}$$

其中 j 为虚数单位，那么：

$$\begin{aligned}
\frac{f+g}{2} * h &= a0 * (b0 + jb1) \\
&= a0 * b0 + ja0 * b1 \\
\frac{f-g}{2j} * h &= a1 * (b0 + jb1) \\
&= a1 * b0 + ja1 * b1 \\
\Rightarrow a * b &= a0 * b0 + (a0 * b1 + a1 * b0) \times base + a1 * b1 \times base^2 \\
&= \mathbf{Re}(\frac{f+g}{2} * h) + (\mathbf{Im}(\frac{f+g}{2} * h) + \mathbf{Re}(\frac{f-g}{2j} * h)) \times base \\
&\quad + \mathbf{Im}(\frac{f-g}{2j} * h) \times base^2
\end{aligned}$$

从而我们只需要计算 $\frac{f+g}{2} * h$ 和 $\frac{f-g}{2j} * h$。令 f 和 g 的离散傅里叶变换分别为 F 和 G，我们进一步发现 $G[k]$ 与 $F[n-k]$ 互为共轭，证明如下：

$$\begin{aligned}
G[k] &= \sum_{i=0}^{n-1} (a0[i] - ja1[i]) e^{-j\frac{2\pi}{n}ki} \\
&= \sum_{i=0}^{n-1} (a0[i] - ja1[i])(\cos(-\frac{2\pi}{n}ki) + j\sin(-\frac{2\pi}{n}ki)) \\
&= \sum_{i=0}^{n-1} (a0[i]\cos\frac{2\pi}{n}ki - a1[i]\sin\frac{2\pi}{n}ki) - j(a0[i]\sin\frac{2\pi}{n}ki + a1[i]\cos\frac{2\pi}{n}ki) \\
F[n-k] &= \sum_{i=0}^{n-1} (a0[i] + ja1[i]) e^{-j\frac{2\pi}{n}(n-k)i} \\
&= \sum_{i=0}^{n-1} (a0[i] + ja1[i]) e^{j\frac{2\pi}{n}ki} \\
&= \sum_{i=0}^{n-1} (a0[i] + ja1[i])(\cos\frac{2\pi}{n}ki + j\sin\frac{2\pi}{n}ki) \\
&= \sum_{i=0}^{n-1} (a0[i]\cos\frac{2\pi}{n}ki - a1[i]\sin\frac{2\pi}{n}ki) + j(a0[i]\sin\frac{2\pi}{n}ki + a1[i]\cos\frac{2\pi}{n}ki)
\end{aligned}$$

比较 $G[k]$ 和 $F[n-k]$ 每一项系数，有 $\mathbf{Re}\{G[k]\} = \mathbf{Re}\{F[n-k]\}, \mathbf{Im}\{G[k]\} = -\mathbf{Im}\{F[n-k]\}$，即二者互为共轭。另一方面，$e^{-j\frac{2\pi}{n}ni} = e^{e^{-j\frac{2\pi}{n}0i}} = 1$，因此 $G[0]$ 与 $F[0]$ 互为共轭。因此，只需求出 F，即可通过取共轭的方式得到 G，这样，我们只需求 f 和 h 的 DFT，求 $\frac{F+G}{2} * H$ 和 $\frac{F-G}{2j} * H$ 的 IDFT，即可算出 $a * b \bmod p$。总共的 DFT 次数被减少到了 4 次。这就是我们最终的任意模数多项式乘法的算法。

4 Chirp-Z 变换

在上一节中，我们在处理一般情况的多项式乘法时，遇到了多项式次数不为 2 的整数次幂的问题。此时我们无法直接求出其对应的离散傅里叶变换，而是采用了高位补 0 的方式规避了这一问题。自然，这在卷积运算中已经足够，然而，假若我们想得到的不是卷积的结果，而是一个任意次数多项式的离散傅里叶变换本身，前述做法是无能为力的。

因此，本节作为全文的拓展部分，将介绍在这一情况下的处理方法。更一般的，我们将介绍一种基于上述多项式乘法的快速离散 Z 变化的实现，即 Chirp-Z 变换，也称 Bluestein 算法。

Definition 4.1 (Chirp-Z 变换)**.** 给出一个多项式 $A(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{C}[x]$ 和 $z \in \mathbb{C} \setminus \{0\}$，求 $A(z^0), A(z^1), A(z^2), \ldots, A(z^{n-1})$。

注意到对于非负整数 k 和 i，我们有：

$$\begin{aligned}\binom{k+i}{2} - \binom{k}{2} - \binom{i}{2} &= \frac{(k+i)(k+i-1)}{2} - \frac{k(k-1)}{2} - \frac{i(i-1)}{2} \\ &= \frac{k^2 + 2ki + i^2 - k - i - (k^2 - k) - (i^2 - i)}{2} \\ &= \frac{2ki}{2} \\ &= ki\end{aligned}$$

因此

$$\begin{aligned}A(z^k) &= \sum_{i=0}^{n-1} a_i z^{ki} \\ &= \sum_{i=0}^{n-1} a_i z^{\binom{k+i}{2} - \binom{k}{2} - \binom{i}{2}} \\ &= z^{-\binom{k}{2}} \sum_{i=0}^{n-1} a_i z^{\binom{k+i}{2} - \binom{i}{2}}\end{aligned}$$

注意到上式是一个差卷积的形式，可令 $F(x) = \sum_i f_{n-1-i} z^{-\binom{n-1-i}{2}} x^i$，其中 $f_i = \begin{cases} a_i & 0 \le i < n \\ 0 & i < 0 \end{cases}$，$G(x) = \sum_i z^{\binom{i}{2}} x^i$，那么：

$$
\begin{aligned}
[x^{n-1+t}](F(x)*G(x)) &= \sum_{i=0}^{n-1+t}([x^{n-1+t-i}]F(x))([x^i]G(x)) \\
&= \sum_{i=0}^{n-1+t} f_{i-t} z^{\binom{i}{2}-\binom{i-t}{2}} \\
&= \sum_{i=t}^{n-1+t} f_{i-t} z^{\binom{i}{2}-\binom{i-t}{2}} \\
&= \sum_{i=0}^{n-1} f_i z^{\binom{i+t}{2}-\binom{i}{2}} \\
&= z^{\binom{t}{2}} \cdot A(z^t)
\end{aligned}
$$

因此我们发现，$F(x)*G(x)$ 中 x^{n-1+t} 项系数恰好是待求的 $A(z^t)$ 乘以 $z^{\binom{t}{2}}$ 的结果。因此，我们可以通过一次卷积（即 F 与 G 的卷积）算出 A(x) 的 Z 变换。同时注意到 $z^{\binom{i+1}{2}} = z^{\binom{i}{2}} \cdot z^i$，因此 $z^{\binom{i}{2}}$ 可以线性递推得到。

综上所述，我们得到了一个通过一次卷积，在 $O(nlogn)$ 时间复杂度内算出一个多项式的 Z 变换的算法，即 Chirp-Z 变换。自然，只需令 $z = e^{-j\frac{2\pi}{n}}$，其结果就是长度为 n 的离散傅里叶变换。

在附录部分，我们结合上一节提出的任意模数多项式乘法，给出了在模 10^9+7 意义下的 Chirp-Z 变换的实现。自然，读者可以轻易的将任意模数多项式乘法替换为一般情况的多项式乘法，将这一实现改为一般意义下的 Chirp-Z 变换，即 Z 变换。

5 测试及结果

在本节中，我们对第三节中提到的实系数多项式乘法的运行速度进行测试，并与 matlab 的 conv 函数的用时进行对比。需要指出的是，matlab 的 conv 函数采用的是 $O(n^2)$ 的朴素实现，因此在大规模的数据下进行对比是不公平且无意义的。因此我们将测试范围限定在 $[5000, 200000]$ 这一范围中，同时每隔 5000 取点作为数据规模进行测试。为了消除 I/O 时间造成的影响，所用的程序均不输出，且不计输入时间。此外，考虑到当数据规模较小时用时也较小，难以区分，因此我们在每个数据规模上测试 10 组数据，取这 10 组数据的总用时作为在这一数据点上的用时。测试结果如下图所示：

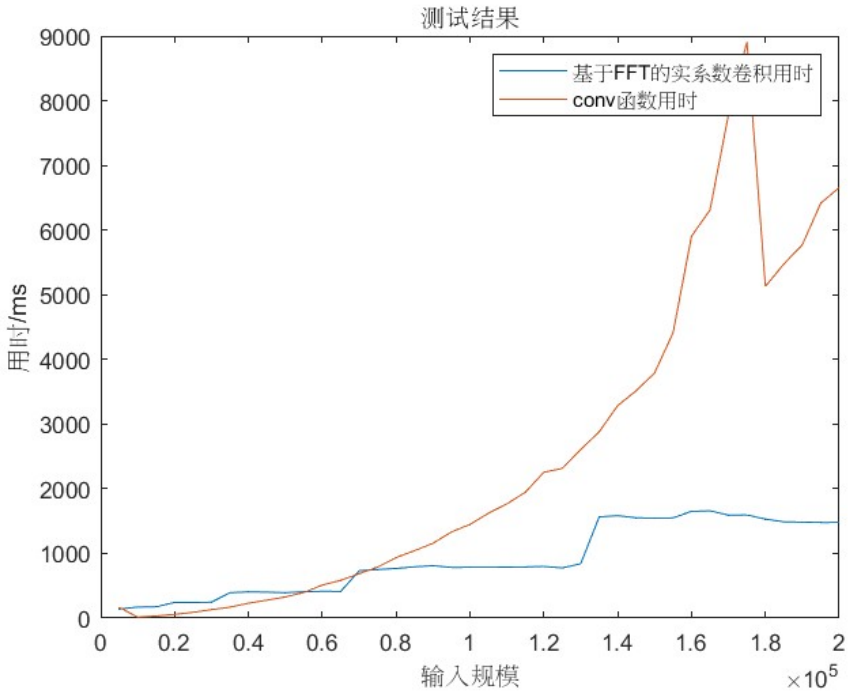


Figure 1: 测试结果

可以发现，当输入规模较大时，基于 FFT 的实系数卷积算法远快于 conv 函数；另一方面，由于在编译时没有设置优化选项，因此 FFT 算法的常数较大，在数据规模较小时没有太大的优势。此外，测试结果与我们的预计符合——conv 函数的用时呈现出明显的平方级别增长，而 FFT 算法则大致符合 $O(nlogn)$ 的时间复杂度。注意到 FFT 算法的用时呈现阶梯状，在 $2^15 = 32768, 2^16 = 65536, 2^17 = 131072$ 附近，用时明显增大，这是因为我们需要将输入的多项式补 0 使其次数为 2 的整数次幂，因此在上述这些点时，进行运算的多项式次数将会翻倍；在其他点时次数不变，从而呈现出阶梯状的特征。

测试所使用的代码在附件中给出。考虑到具体的用时与测试环境相关，读者也可使用提供的测试代码验证这一结果的正确性。

6 Appendix: Source Code (in C++)

6.1 快速傅里叶变换及利用 FFT 的快速卷积

```cpp
#include <bits/stdc++.h>

using namespace std;

const int MAX_N = 1 << 21;
const double Pi = acos(-1);

struct Complex{
    double re, im;

    Complex () : re(0), im(0) {}
    Complex (double a, double b) : re(a), im(b) {}

    inline Complex operator + (const Complex &a) const {return Complex(re + a.re, im + a.im);}
    inline Complex operator - (const Complex &a) const {return Complex(re - a.re, im - a.im);}
    inline Complex operator * (const Complex &a) const {return Complex(re * a.re - im * a.im, re * a.im + im * a.re);}
    inline Complex& operator *= (const Complex &a) {return *this = *this * a;}
};

const Complex* fft_init_w(int n) {
    static Complex* w[MAX_N];
    if (w[n] == NULL) {
        Complex *wn = new Complex[n];
        wn[0].re = 1;
        if (n > 1) {
            const Complex w0 = Complex(cos(Pi / n), sin(Pi / n));
            const Complex *wm = fft_init_w(n >> 1);
            for (int i = 0; i < n; i += 2)
                wn[i + 1] = (wn[i] = wm[i >> 1]) * w0;
        }
        w[n] = wn;
    }
    return w[n];
```

```
34 }
35
36 void fft(Complex *a, int n, int ty) { // ty = 1, DFT; ty = -1,
       IDFT
37     for (int i = 0, j = 0; i < n; i ++) {
38         if (i < j) swap(a[i], a[j]);
39         for (int k = n >> 1; (j ^= k) < k; k >>= 1);
40     }
41     for (int i = 1; i < n; i <<= 1) {
42         const Complex *w = fft_init_w(i);
43         for (int j = 0; j < n; j += i << 1)
44             for (int k = j; k < j + i; k ++) {
45                 const Complex t1 = a[k], t2 = a[k + i] * w[k -
                       j];
46                 a[k] = t1 + t2; a[k + i] = t1 - t2;
47             }
48     }
49     if (ty == 1) {
50         reverse(a + 1, a + n);
51         return ;
52     }
53     const double inv = 1.0 / n;
54     for (int i = 0; i < n; i ++) {
55         a[i].re *= inv;
56         a[i].im *= inv;
57     }
58 }
59
60 int main() {
61         int N, M;
62         static double a[MAX_N], b[MAX_N];
63         scanf("%d%d", &N, &M);
64         for (int i = 0; i < N; i ++) scanf("%lf", a + i);
65         for (int i = 0; i < M; i ++) scanf("%lf", b + i);
66         int len;
67         for (len = 1; len < N + M; len <<= 1);
68         static Complex A[MAX_N];
69         for (int i = 0; i < N && i < M; i ++) {
70                 A[i].re = a[i] + b[i];
71                 A[i].im = a[i] - b[i];
72         }
73         for (int i = M; i < N; i ++) {
74                 A[i].re = a[i]; A[i].im = a[i];
```

```
        }
        for (int i = N; i < M; i ++) {
                A[i].re = b[i]; A[i].im = -b[i];
        }
        fft(A, len, 1);
        for (int i = 0; i < len; i ++) A[i] *= A[i];
        fft(A, len, -1);
        for (int i = 0; i < N + M - 1; i ++) {
        printf("%.4lf%c", A[i].re / 4, "\n "[i < N + M - 2]);
        }
    return 0;
}
```

6.2 Chirp-Z 变换

```
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;

const int MAX_N = 1 << 21;
const int P = 1000000000 + 7;
const double Pi = acos(-1);

int N, M, c, a[MAX_N], a0[MAX_N], b0[MAX_N];

struct Complex{
        double re, im;

        Complex () : re(0), im(0) {}
        Complex (double a, double b) : re(a), im(b) {}

        inline Complex operator + (const Complex &a) const {
           return Complex(re + a.re, im + a.im);}
        inline Complex operator - (const Complex &a) const {
           return Complex(re - a.re, im - a.im);}
        inline Complex operator * (const Complex &a) const {
           return Complex(re * a.re - im * a.im, re * a.im +
           im * a.re);}
        inline Complex& operator *= (const Complex &a) {return
            *this = *this * a;}
};
```

```
24
25 const Complex* fft_init_w(int n) {
26     static Complex *w[MAX_N];
27     if (w[n] == NULL) {
28         Complex *wn = new Complex[n];
29         wn[0].re = 1;
30         if (n > 1) {
31             const Complex w0 = Complex(cos(Pi / n), sin(Pi / n
                   ));
32             const Complex *wm = fft_init_w(n >> 1);
33             for (int i = 0; i < n; i += 2)
34                 wn[i + 1] = (wn[i] = wm[i >> 1]) * w0;
35         }
36         w[n] = wn;
37     }
38     return w[n];
39 }
40
41 void fft(Complex *a, int n, int ty) {
42     for (int i = 0, j = 0; i < n; i ++) {
43         if (i < j) swap(a[i], a[j]);
44         for (int k = n >> 1; (j ^= k) < k; k >>= 1);
45     }
46     for (int i = 1; i < n; i <<= 1) {
47         const Complex *w = fft_init_w(i);
48         for (int j = 0; j < n; j += i << 1)
49             for (int k = j; k < j + i; k ++) {
50                 const Complex t1 = a[k], t2 = a[k + i] * w[k -
                       j];
51                 a[k] = t1 + t2; a[k + i] = t1 - t2;
52             }
53     }
54     if (ty == 1) return ;
55     reverse(a + 1, a + n);
56     const double inv = 1.0 / n;
57     for (int i = 0; i < n; i ++) {
58         a[i].re *= inv; a[i].im *= inv;
59     }
60 }
61
62 void mul(int *a, int *b, int *res, int n, int m, int len, int
       p) {
63     static Complex A[MAX_N], B[MAX_N], c0[MAX_N], c1[MAX_N];
```

```
64      for (int i = 0; i < n; i ++) A[i] = Complex(a[i] & 32767,
            a[i] >> 15);
65      for (int i = 0; i < m; i ++) B[i] = Complex(b[i] & 32767,
            b[i] >> 15);
66      fft(A, len, 1); fft(B, len, 1);
67      for (int i = 0; i < len; i ++) {
68          const int conj = (len - i) & (len - 1);
69          c0[i] = Complex(0.5 * (A[i].re + A[conj].re), 0.5 * (A
                [i].im - A[conj].im)) * B[i];
70          c1[i] = Complex(0.5 * (A[i].im + A[conj].im), 0.5 * (A
                [conj].re - A[i].re)) * B[i];
71      }
72      for (int i = 0; i < len; i ++) A[i] = B[i] = Complex();
73      fft(c0, len, -1); fft(c1, len, -1);
74      for (int i = 0; i < n + m; i ++) {
75          ll c00 = c0[i].re + 0.5, c01 = c0[i].im + 0.5, c10 =
                c1[i].re + 0.5, c11 = c1[i].im + 0.5;
76          res[i] = (c00 + ((c01 + c10) % p << 15) + (c11 % p <<
                30)) % p; c0[i] = c1[i] = Complex();
77      }
78  }
79
80  void exgcd(int a, int b, int &x, int &y) {
81      if (!b) {
82          x = 1; y = 0; return ;
83      }
84      exgcd(b, a % b, y, x);
85      y -= a / b * x;
86  }
87
88  int getInv(int x) {
89      int inv = 0;
90      exgcd(x, P, inv, x);
91      return inv < 0 ? inv + P : inv;
92  }
93
94  int main() {
95      cin >> N >> c >> M;
96
97      for (int i = 0; i < N; i ++) cin >> a[i];
98
99      int ci_neg = 1, ci_pos = 1, ci = 1, ci_inv = 1, len = 1;
100     const int invc = getInv(c);
```

```
101
102         while (len < N + M) len <<= 1;
103
104         for (int i = 0; i < N + M; i ++) {
105             b0[i] = ci_pos;
106             if (i < N) a0[N - i] = 1ll * a[i] * ci_neg % P;
107             ci_pos = 1ll * ci_pos * ci % P;
108             ci_neg = 1ll * ci_neg * ci_inv % P;
109             ci = 1ll * ci * c % P;
110             ci_inv = 1ll * ci_inv * invc % P;
111         }
112
113         mul(a0, b0, a0, N + 1, N + M, len, P);
114
115         ci_inv = 1; ci_neg = 1;
116         for (int i = 0; i < M; i ++) {
117             a0[i + N] = 1ll * a0[i + N] * ci_neg % P;
118             cout << a0[i + N] << "\n␣"[i < M - 1];
119             ci_neg = 1ll * ci_neg * ci_inv % P;
120             ci_inv = 1ll * ci_inv * invc % P;
121         }
122
123         return 0;
124 }
```